

DriveSynergyBot

System Documentation

Purpose	Resume matchmaking Telegram bot for private business communities
Architecture	Python async · Weaviate · MongoDB · MinIO · OpenRouter LLM
Framework	python-telegram-bot v21 · Pydantic Settings · Docker Compose
Languages	English · Russian (auto-detected, user-switchable)

This document describes the component architecture and main interaction flows of DriveSynergyBot. Each section contains a rendered diagram and a written description.

1. System Overview

DriveSynergyBot is a Telegram bot that serves as a professional matchmaking platform for closed business communities. Members register by uploading a resume (PDF or DOCX). The bot extracts a structured profile using an LLM, embeds it into a vector space, and stores it alongside the original file. Any registered member can search for collaborators using natural language; the bot returns ranked result cards with contact deep-links.

Key Characteristics

Entry point	Telegram group chat or private chat
Registration	FSM ConversationHandler — upload file → preview → confirm
Search	Weaviate hybrid search (vector + BM25) → LLM re-rank
LLM	OpenRouter (default: anthropic/claude-3-haiku); configurable
Embeddings	sentence-transformers all-MiniLM-L6-v2 — 384-dim, runs locally
Persistence	MongoDB (registry) · Weaviate (vectors) · MinIO (files)
i18n	English + Russian — auto-detected from Telegram locale
Deployment	Docker Compose; Azure migration path documented in README

2. Architecture & Flow Diagrams

2.1 System Architecture — Component Diagram

Shows all system components, their responsibilities, and data flows. The Telegram Bot layer is the entry point; it routes user commands to specialised handlers. The Pipeline layer transforms raw resume files into structured vector profiles. The Storage layer provides three complementary stores: MongoDB for user registry, Weaviate for semantic search, and MinIO for original files. The LLM layer (OpenRouter) is used for extraction, ranking, and community summaries.

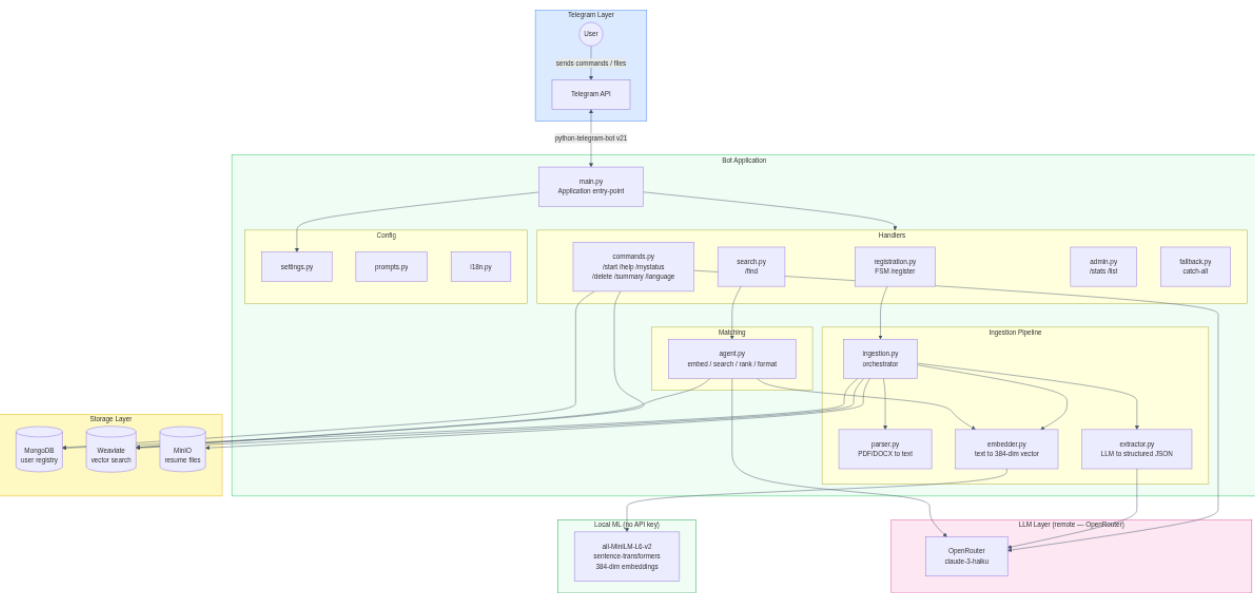


Figure 2.1: System Architecture — Component Diagram

2.2 Sequence Diagram — Registration Flow (/register)

A user triggers /register which starts a ConversationHandler FSM. The bot waits for a document upload, runs the ingestion pipeline (parse → LLM extraction → embedding), shows a profile preview, then waits for confirmation. On confirmation the profile is persisted to all three stores. The same flow handles re-registration (upsert).

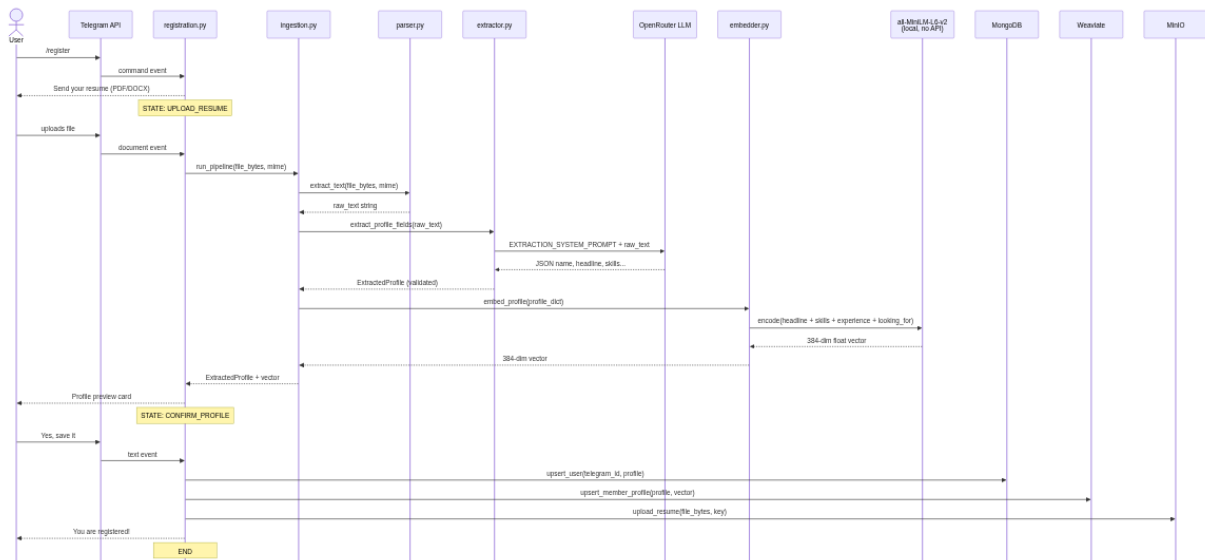


Figure 2.2: Sequence Diagram — Registration Flow (/register)

2.3 Sequence Diagram — Search Flow (/find)

A registered user invokes /find (optionally with an inline query). If no query is provided, the FSM waits for text input. The matching agent embeds the query, runs Weaviate hybrid search (vector + BM25), sends the top candidates to the LLM for re-ranking with match reasoning, then formats result cards with contact deep-links.

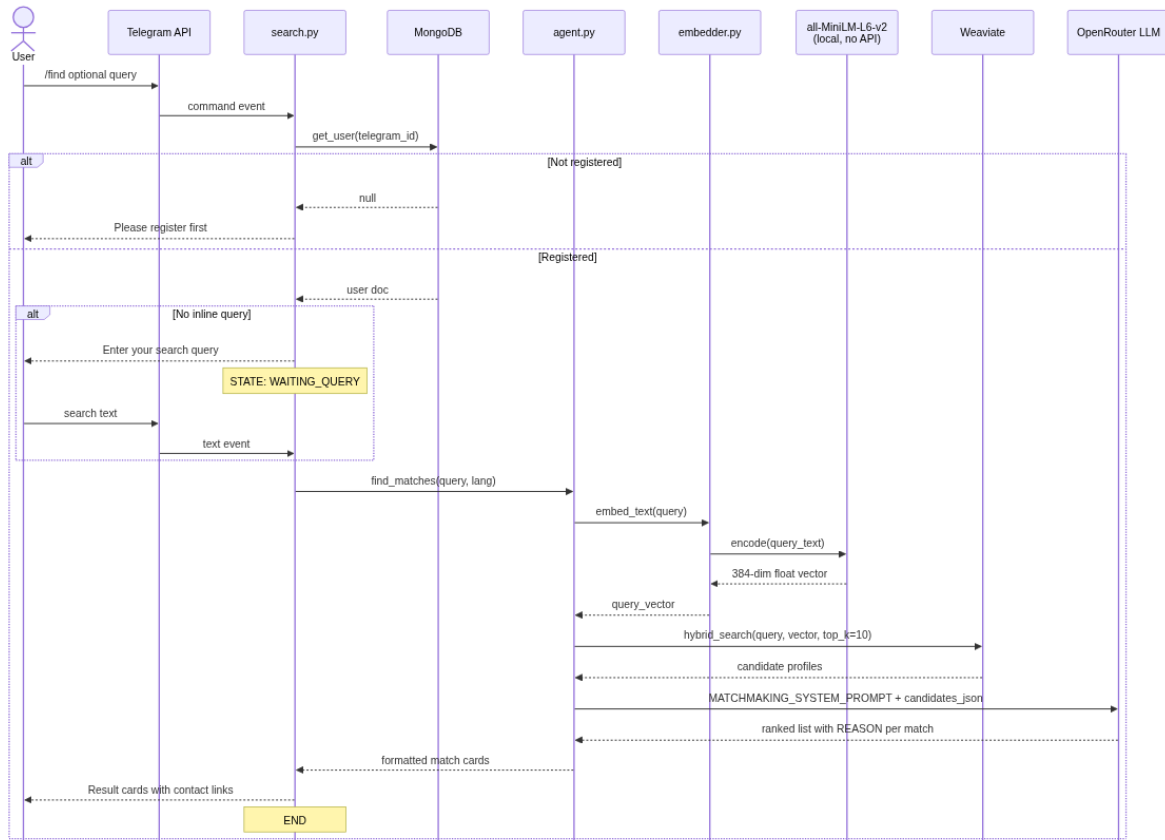


Figure 2.3: Sequence Diagram — Search Flow (/find)

2.4 Sequence Diagram — Community Summary (/summary)

The /summary command provides a community overview without exposing individual profiles. Instead of sending raw profile data to the LLM, the handler first aggregates data locally (skill/industry frequency tables, looking-for samples) and sends only the compact statistics, significantly reducing token usage.

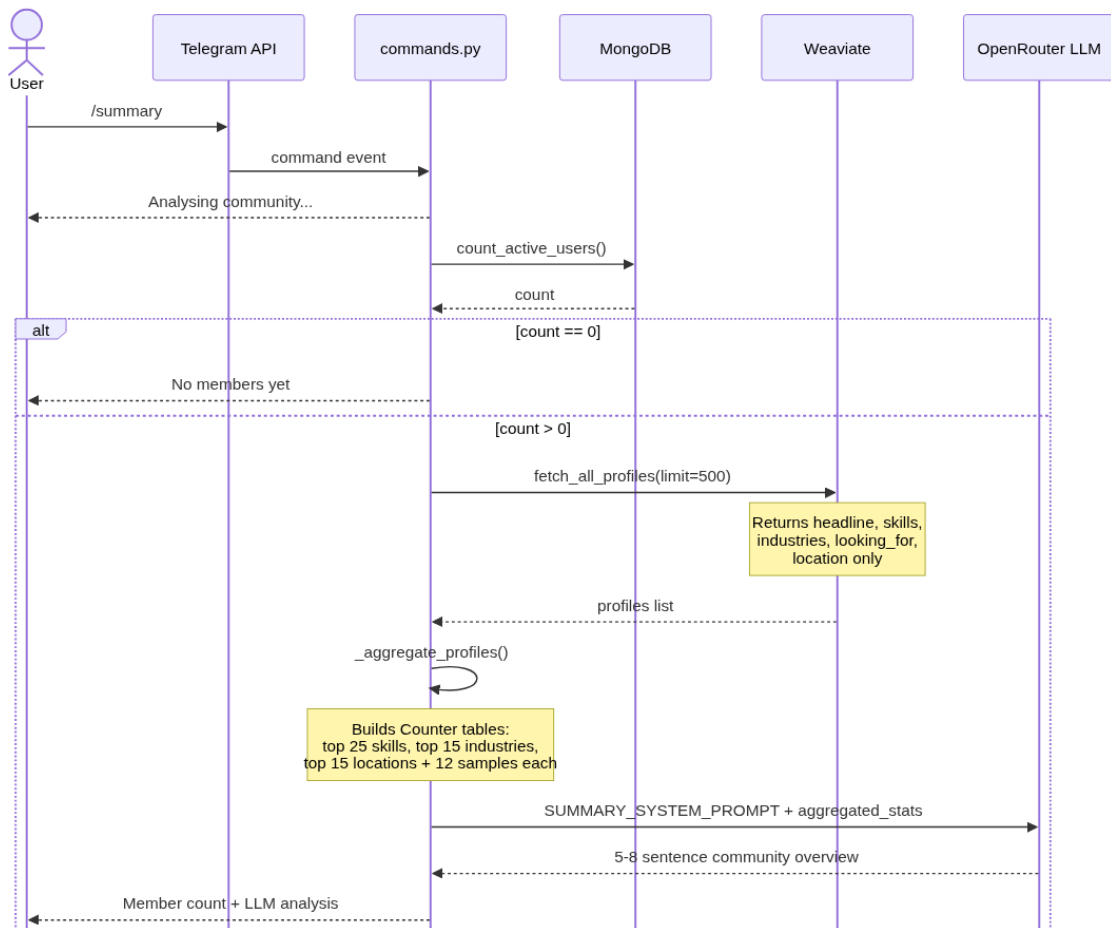


Figure 2.4: Sequence Diagram — Community Summary (/summary)

2.5 User State Diagram

A user starts as Unknown and moves to Registered after completing the registration flow. They can re-register at any time (updating their profile) or delete their profile to return to the Unknown state. Registered users have full access to /find, /mystatus, and /delete. Unknown users are directed to /register.

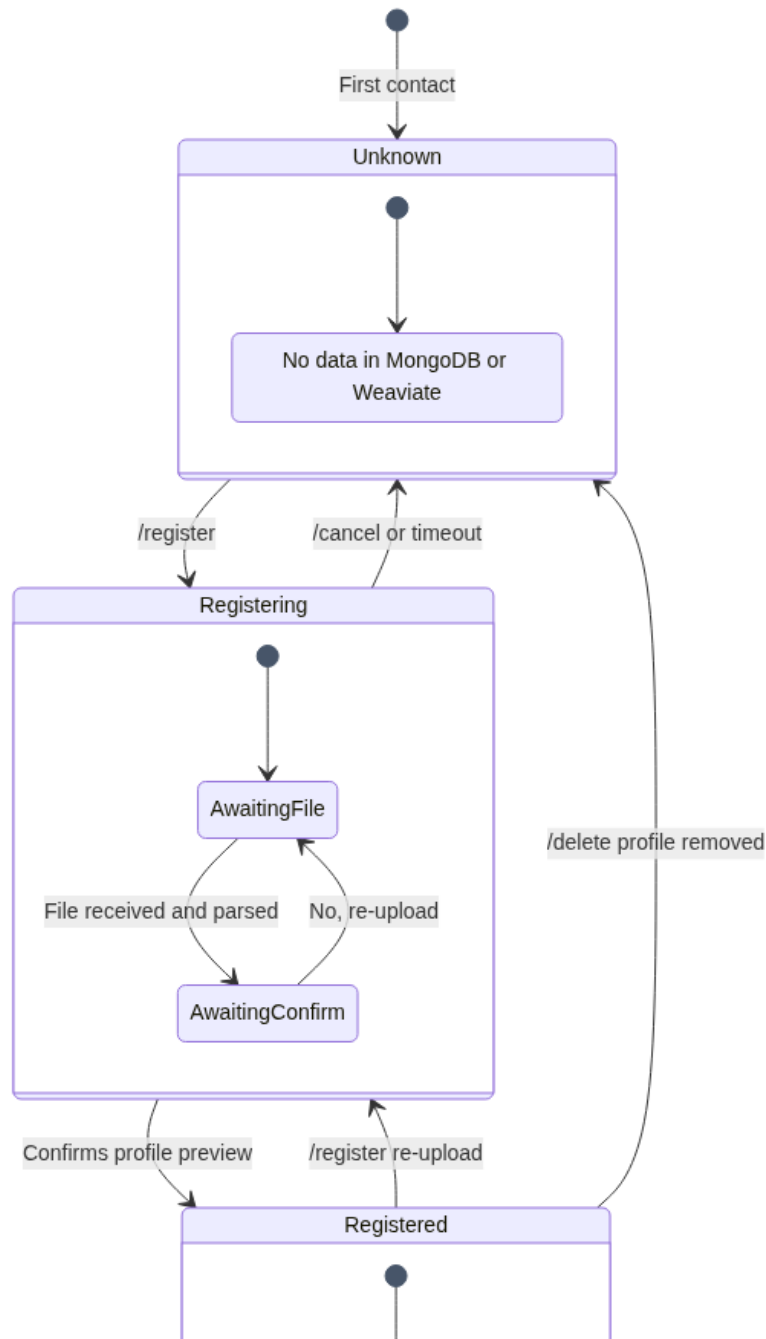


Figure 2.5: User State Diagram

2.6 Activity Diagram — Registration

Detailed activity flow for the /register command showing all decision points: whether the user is already registered (upsert vs insert), file format validation, LLM extraction success/failure, and user confirmation. The pipeline runs entirely inside the handler before any data is persisted.

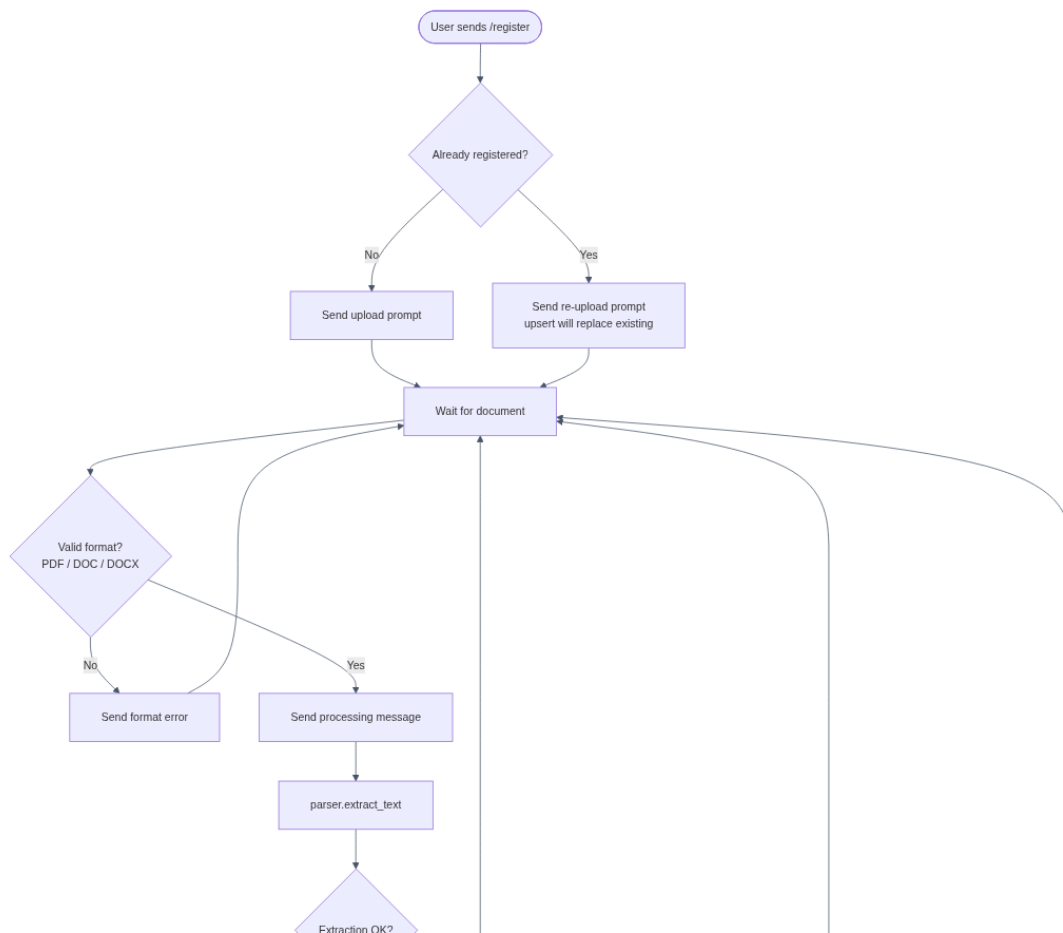


Figure 2.6: Activity Diagram — Registration

2.7 Activity Diagram — Search (/find)

Activity flow for the /find command showing the registration check, optional query collection via FSM, hybrid search in Weaviate, LLM re-ranking, and result formatting. If no candidates are found by Weaviate, or the LLM determines there are no matches, the user receives a descriptive empty-result message.

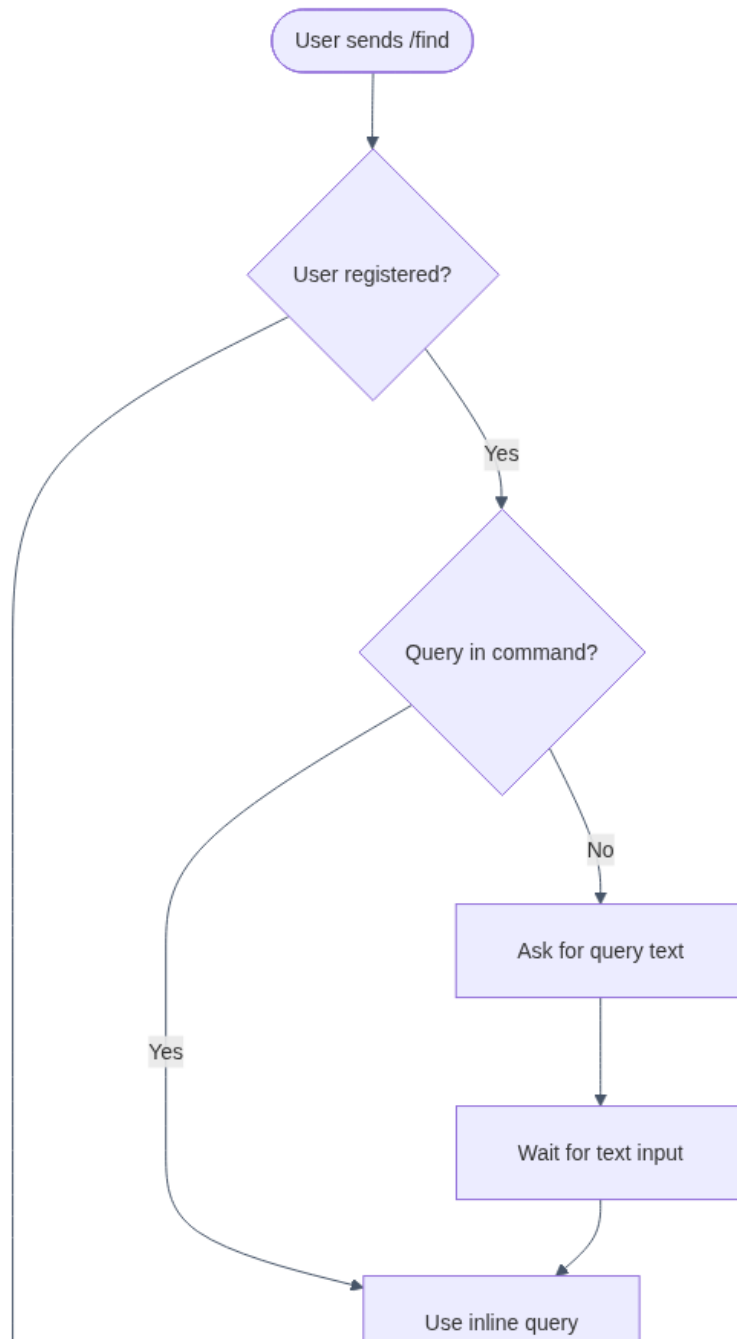


Figure 2.7: Activity Diagram — Search (/find)

2.8 i18n Language Resolution Flow

Every handler calls `get_lang()` before sending any message. The function checks multiple sources in priority order: in-memory `user_data` cache (fastest), MongoDB persisted preference, Telegram's reported locale, then falls back to English. The `/language` command and inline keyboard let users override at any time.

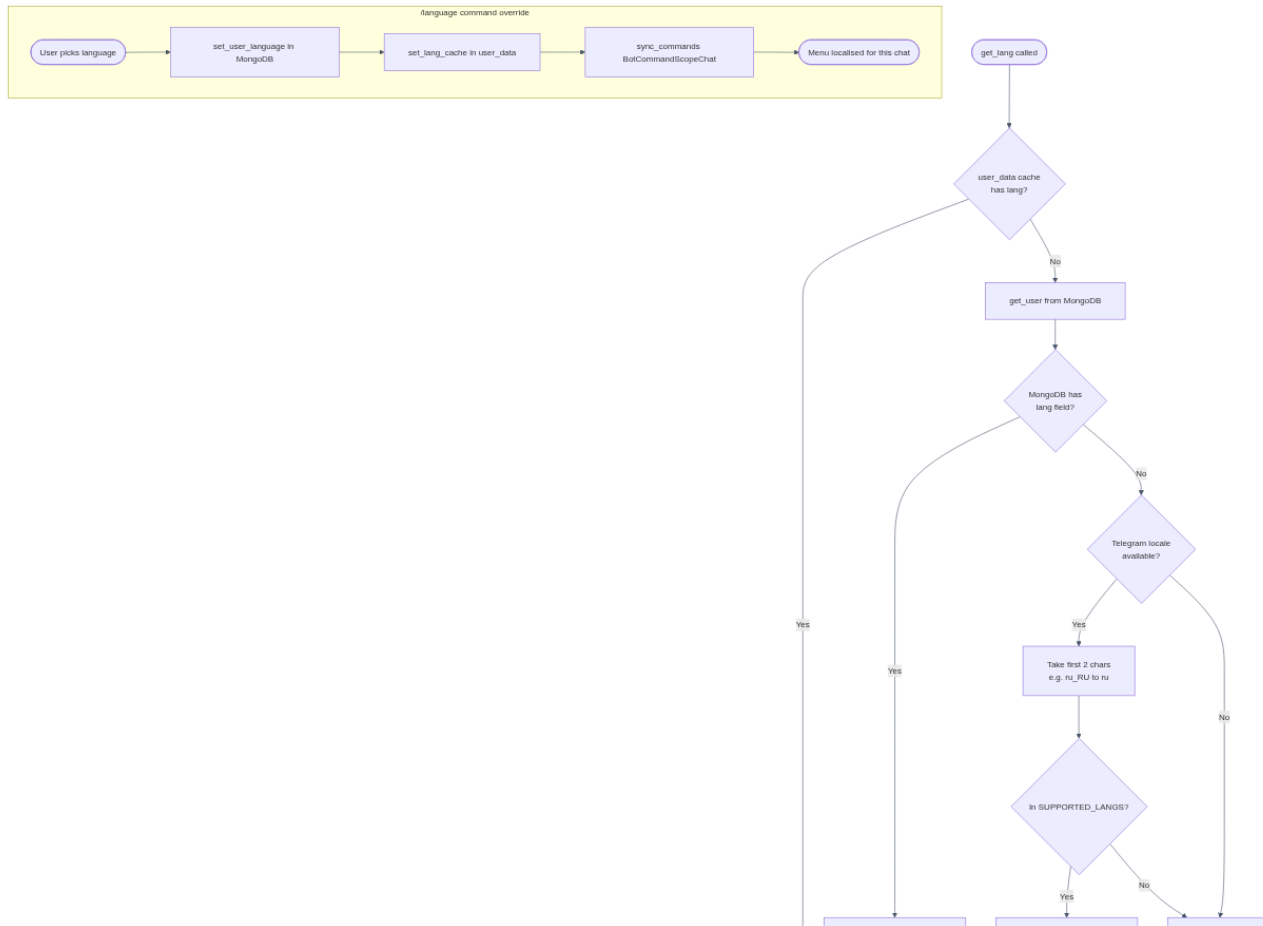


Figure 2.8: i18n Language Resolution Flow

3. Component Descriptions

main.py — Application Entry Point

Builds the python-telegram-bot Application object, registers all ConversationHandlers and command handlers in the correct priority order, sets the global (English) command menu, and starts polling. Also runs the post-init hook that ensures all storage schemas exist.

handlers/registration.py — Registration FSM

A ConversationHandler with two states: UPLOAD_RESUME (waiting for a document) and CONFIRM_PROFILE (waiting for yes/no after preview). Handles re-registration (upsert) transparently — deletes old Weaviate object and overwrites MinIO file.

handlers/search.py — Search Handler

ConversationHandler for /find. If no inline query is provided the FSM waits for text input (120 s timeout). Delegates all search logic to matching/agent.py. Checks registration status before running any search.

handlers/commands.py — Basic Commands

/start sets the per-chat command menu and sends a welcome card. /mystatus shows stored name, username, and registration dates. /delete removes all stored data. /summary pre-aggregates all profiles into frequency tables before the LLM call (token-efficient). /language shows an inline keyboard picker.

handlers/admin.py — Admin Commands

/stats returns active user count (Telegram ID must be in ADMIN_TELEGRAM_IDS). /list returns a paginated table of all registered members.

pipeline/parser.py — Document Parser

Extracts plain text from PDF bytes (pymupdf/fitz) or DOCX bytes (python-docx). Legacy .doc files handled via antiword. Returns cleaned, whitespace-normalised text.

pipeline/extractor.py — LLM Profile Extractor

Single OpenRouter call using EXTRACTION_SYSTEM_PROMPT. Returns a Pydantic-validated ExtractedProfile: full_name, headline, skills[], industries[], experience, looking_for, location. Logs model, token counts, and latency.

pipeline/embedder.py — Vector Embedder

Loads all-MiniLM-L6-v2 once at startup (~90 MB). Produces a 384-dim vector from headline + skills + experience + looking_for. Runs locally — no API needed.

pipeline/ingestion.py — Ingestion Orchestrator

Coordinates parser → extractor → embedder, writes to all three stores. Returns ExtractedProfile for preview rendering.

matching/agent.py — Matchmaking Agent

Embeds the query, runs Weaviate hybrid search, sends top candidates to LLM, parses structured output (RANK/NAME/HEADLINE/REASON/SKILLS/TELEGRAM_ID blocks), formats result cards with tg:// contact deep-links. Language-aware.

storage/weaviate_client.py — Weaviate Client

Wraps synchronous Weaviate v4 Python client in `asyncio.run_in_executor`. Provides: `ensure_schema`, `upsert_member_profile`, `hybrid_search`, `fetch_all_profiles` (compact), `delete_member_profile`.

storage/mongo_client.py — MongoDB Client

Motor async driver. User registry CRUD plus get/set language preference. Indexed on `telegram_id`.

storage/minio_client.py — MinIO Client

Async wrappers over synchronous MinIO SDK. Files keyed as `resumes/{telegram_id}/{filename}`. Deleted on user profile deletion.

config/settings.py — Configuration

Pydantic Settings from `.env` + `.env.bot`. Custom validator for `ALLOWED_GROUP_ID` accepts empty string as `None`.

config/prompts.py — LLM Prompts

Single source for all LLM prompt strings: extraction, matchmaking (with language instruction), and community summary templates.

config/i18n.py — Internationalisation

`SUPPORTED_LANGS`, per-language BotCommand lists, and `_STRINGS` dict with all user-facing copy. `t(key, lang, **kwargs)` with English fallback.

4. Data Models

MongoDB — users collection

Field	Type	Description
telegram_id	int	Telegram user ID — primary key, indexed
telegram_username	str null	@handle from Telegram
full_name	str	Extracted from resume
registered_at	datetime	First registration timestamp
updated_at	datetime	Last profile update timestamp
profile_status	str	"active" or "deleted"
lang	str	"en" or "ru" — language preference
minio_key	str	Path in MinIO: resumes/{telegram_id}/{file}
weaviate_uuid	str	UUID of the Weaviate MemberProfile object

Weaviate — MemberProfile collection

Vector is the 384-dim embedding of: headline + skills + experience + looking_for.

Property	Type	Description
telegram_id	INT	Links back to MongoDB user
full_name	TEXT	Person's full name
headline	TEXT	One-sentence professional summary
skills	TEXT[]	Array of skill tags
industries	TEXT[]	Array of industry tags
experience	TEXT	2-3 sentence career narrative
looking_for	TEXT	Collaboration or opportunity statement
location	TEXT	City/country or Remote
resume_text	TEXT	Full extracted text for RAG
[vector]	float[384]	Embedding for hybrid search

5. Bot Commands

Command	Description	Access
/start	Welcome message; sets per-chat command menu	All
/register	Upload resume (PDF/DOC/DOCX); re-upload updates profile	All
/find [query]	Search for matching profiles by natural language	Registered
/mystatus	Show your current profile summary	All
/delete	Remove your profile from all stores	Registered
/summary	Community overview — member count + LLM analysis	All
/language	Switch bot language via inline keyboard	All
/cancel	Cancel any active conversation flow	All
/help	List all commands	All
/stats	Total member count	Admin only
/list	Paginated list of all registered members	Admin only

6. Future Work

Voice Search (Planned)

Voice messages → Whisper API speech-to-text → LLM cleanup pass → existing /find pipeline. Azure AI Speech replaces Whisper in production. See README for implementation notes.

Azure Production Deployment (TBD)

matchbot → Azure Container Apps, MongoDB → Cosmos DB for MongoDB, MinIO → Azure Blob Storage, LLM → Azure OpenAI. Weaviate: AKS or Azure AI Search evaluation pending. CI/CD via GitHub Actions → ACR → Container Apps.